

Student Fest 2026 — разбор задач

Полный текстовый разбор задач А–J

Содержание

А. Проблемы со связью	2
В. Гномья бюрократия	3
С. Секретная пара	5
Д. Организация чемпионата	6
Е. Потеряшка	7
Ф. Поиск составителей	9
Г. Разбиение на отрезки	10
Н. Шахматные проказы	12
І. Экзамен	13
Ј. Тактический симулятор	15

А. Проблемы со связью

Одну строку отправили k раз. При передаче номер каждой буквы мог измениться не более чем на d , без перехода через конец алфавита. Известна сумма номеров исходных букв C . Нужно восстановить лексикографически минимальную строку.

Допустимые значения в каждой позиции

Пусть $t_{j,i}$ — номер буквы в позиции i копии j , а s_i — искомый номер исходной буквы. Каждая копия задаёт ограничение $t_{j,i} - d \leq s_i \leq t_{j,i} + d$. Пересечём эти ограничения и диапазон алфавита:

$$L_i = \max\left(1, \max_j(t_{j,i} - d)\right), \quad R_i = \min\left(26, \min_j(t_{j,i} + d)\right).$$

Если для какой-то позиции $L_i > R_i$, ответа нет. Иначе минимальная возможная сумма равна $\sum_i L_i$, максимальная — $\sum_i R_i$. Между ними можно получить любую целую сумму: каждую букву разрешено увеличивать по единице в пределах её интервала. Поэтому оставшееся условие существования ответа:

$$\sum_i L_i \leq C \leq \sum_i R_i.$$

Лексикографический минимум

Сначала положим $s_i = L_i$. Не хватает ещё $\Delta = C - \sum_i L_i$ единиц. Будем идти справа налево и прибавлять к очередной букве $\min(\Delta, R_i - L_i)$, уменьшая Δ на эту величину.

Так мы откладываем все вынужденные увеличения как можно правее. Если в другой строке левая позиция увеличена, хотя справа ещё есть свободное место, можно перенести единицу вправо: сумма и допустимость сохранятся, а строка станет лексикографически меньше. Значит, в минимальном ответе правые позиции должны использоваться раньше левых — именно так работает наш алгоритм.

Например, для трёх интервалов $[1, 3]$ и суммы 6 начинаем с `aaa`, добавляем две единицы в последнюю позицию и одну в предпоследнюю. Получаем `abc`.

Время — $O(kn)$, память — $O(n)$: копии можно обрабатывать по одной.

В. Гномья бюрократия

В корневом дереве лист хранит значение C_v , а внутренняя вершина вычисляет

$$\text{ID}_v = A_v \sum_{u \in \text{ch}(v)} \text{ID}_u + B_v \pmod{10^9 + 7}.$$

После изменения одного параметра нужно узнать значение корня. Обычный пересчёт всех предков может занимать $O(n)$: дерево способно оказаться длинной цепочкой. Нам поможет условие о любых 151 вершине.

Ветвлений мало

Никакие два листа не являются предками друг друга. Значит, листьев не больше 150 — иначе можно было бы выбрать 151 лист и нарушить условие. Обозначим их число через L .

Вершин хотя бы с двумя детьми не больше $L - 1$. Это следует, например, из равенства

$$\sum_{v: \deg^+(v) > 0} (\deg^+(v) - 1) = L - 1,$$

где $\deg^+(v)$ — число детей. Каждая вершина-ветвление вносит в сумму хотя бы единицу, а вершина с одним ребёнком — ноль.

Оставим листья и ветвления, а пути из вершин с одним ребёнком сожмём. Получится дерево из $O(L)$ вершин. Если над его верхней вершиной есть цепочка до исходного корня, будем хранить её отдельно и применять последней.

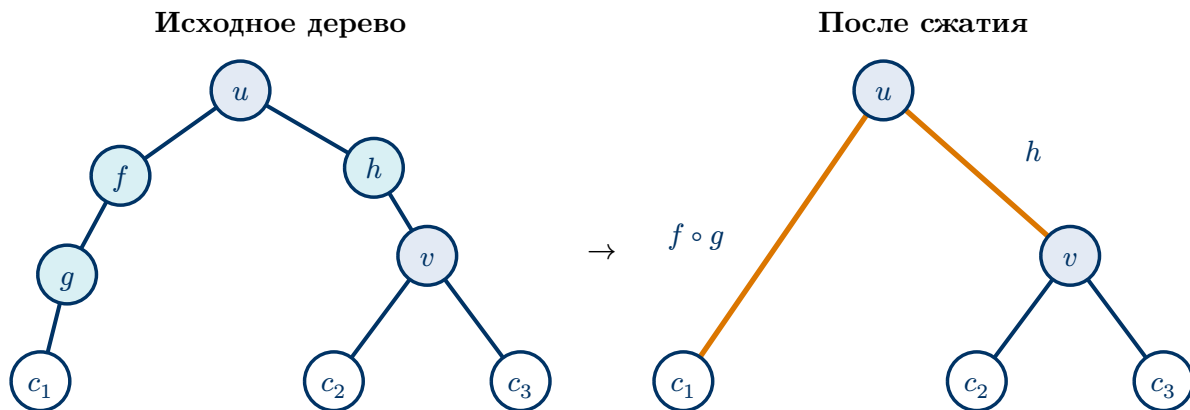


Рис. 1. Голубые вершины заменяем композициями на рёбрах. Остаются три листа и два ветвления.

Что хранить вместо цепочки

Вершина с одним ребёнком действует на его значение как функция $f_v(x) = A_v x + B_v$. Композиция таких функций снова линейна:

$$f(g(x)) = a(cx + d) + b = acx + ad + b,$$

если $f(x) = ax + b$, $g(x) = cx + d$. Поэтому всю цепочку можно заменить двумя коэффициентами её композиции. Все операции выполняются по модулю $10^9 + 7$.

Для каждой цепочки построим дерево отрезков. Функции в листьях расположим сверху вниз. Если левый сын хранит композицию F верхней части цепочки, а правый — композицию G нижней, родитель хранит $F \circ G$: сначала значение проходит нижнюю часть, затем верхнюю. Для пар коэффициентов это операция

$$(a, b) \circ (c, d) = (ac, ad + b).$$

Она ассоциативна, но не коммутативна: порядок менять нельзя. Нейтральный элемент — $(1, 0)$, то есть функция $x \mapsto x$.

После изменения A_v или B_v внутри цепочки обновляется один лист дерева отрезков и $O(\log n)$ его предков. Композиция всей цепочки всегда лежит в корне этого дерева, поэтому получать её отдельным запросом не нужно.

Как обработать изменение

Если изменена вершина цепочки, обновим её композицию. Если изменён лист или вершина-ветвление, просто сохраним новый параметр. Затем пересчитаем всё сжатое дерево снизу вверх: к значению каждого ребёнка применим композицию цепочки на соответствующем ребре, сложим результаты и используем формулу вершины. В конце применим цепочку над верхней вершиной, если она есть.

На сжатом дереве всего $O(L)$ вершин и рёбер, поэтому этот пересчёт дешёвый. Суммарная длина всех цепочек не превосходит n .

Построение — $O(n)$, один запрос — $O(\log n + L)$, где $L \leq 150$. Общая память — $O(n)$.

С. Секретная пара

Из известных пар (X_i, Y_i) выбрана одна. Ваня знает её первую координату, Дания — вторую. В каждом раунде они одновременно называют пару, если могут определить её однозначно. После общего молчания ведущий может исключить ещё одну пару. Нужно найти первый раунд ответа и определить, кто ответит.

Двудольный граф возможных пар

Создадим левую долю из значений X , правую — из значений Y . Каждая пара задаёт ребро. Будем поддерживать граф пар, которые ещё возможны с точки зрения общедоступной информации.

Ваня знает левый конец секретного ребра. Он может ответить ровно тогда, когда степень этого конца равна единице: остаётся единственный вариант второй координаты. Аналогично, Дания отвечает при степени правого конца, равной единице.

Если оба промолчали, секретное ребро не могло быть инцидентно ни одной вершине степени 1. Иначе хотя бы один игрок ответил бы. Следовательно, общее молчание позволяет удалить сразу все такие рёбра. Это не просто отсутствие ответа, а новая информация для обоих игроков.

Удаляем по одному слою за раунд

В начале каждого раунда проверяем степени концов настоящего секретного ребра. Если одна или обе равны 1, получаем ответ **VANYA**, **DANYA** или **BOTH**. Иначе выполняем два действия:

1. Удаляем рёбра при вершинах, имевших степень 1 в начале раунда.
2. Если для этого раунда есть подсказка, удаляем указанное в ней ребро.

Только после этого начинается следующий раунд. Вершины, ставшие листьями в процессе удаления, относятся уже к следующему слою: игроки записали ответы одновременно и не могли использовать ещё не опубликованное молчание друг друга. Будущие подсказки тоже нельзя применять заранее.

Например, пусть возможны пары $(1, 1)$, $(2, 1)$, $(2, 2)$, а секрет — $(2, 1)$. В первом раунде у обоих игроков по два варианта. Их молчание исключает $(1, 1)$ и $(2, 2)$, так как у этих рёбер есть концы степени 1. Во втором раунде оба знают единственную оставшуюся пару.

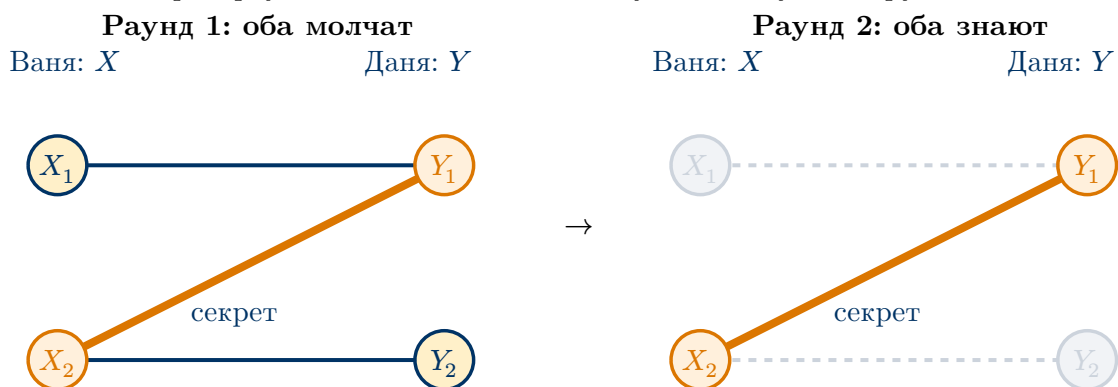


Рис. 2. Секрет — оранжевое ребро (X_2, Y_1) . Общее молчание исключает пунктирные альтернативы.

Линейная реализация

Поддерживаем степени, отметки удалённых рёбер и два списка листьев: текущий и следующий слой. Чтобы быстро находить единственное ребро вершины степени 1, можно хранить XOR номеров её активных рёбер. При удалении ребра меняем степень и XOR обоих концов. Повторное удаление уже неактивного ребра ничего не делает; вершину из очереди, чья степень уже стала нулевой, пропускаем.

Если слоёв больше нет, но остались подсказки, продолжаем применять их по одной после соответствующих раундов. Если нет ни новых листьев, ни будущих подсказок, граф больше не изменится и ответ — **NEVER**.

Каждое ребро удаляется не более одного раза, каждая вершина становится листом не более одного раза. Время — $O(m + q)$, память — $O(m + q)$.

Д. Организация чемпионата

Дан ациклический ориентированный граф зависимостей. Каждая работа занимает один день, ребро $u \rightarrow v$ требует выполнить u строго раньше v . Параллельных работ может быть сколько угодно. Для каждой вершины нужно найти число дней, в которые её можно поставить в расписании минимальной длительности.

Самый ранний и самый поздний день

В топологическом порядке посчитаем ранние дни:

$$e_v = 1 + \max_{u \rightarrow v} e_u.$$

Максимум по пустому множеству считаем равным нулю. Величина e_v — длина самого длинного пути, заканчивающегося в v , в вершинах. Раньше этого дня работу выполнить нельзя. С другой стороны, назначение всем работам дней e_v удовлетворяет зависимостям. Поэтому минимальная длительность равна $T = \max_v e_v$.

В обратном топологическом порядке найдём длину самого длинного пути, начинающегося в вершине:

$$h_v = 1 + \max_{v \rightarrow u} h_u.$$

После работы v нужны ещё хотя бы $h_v - 1$ дней, значит, её поздний день равен

$$l_v = T - h_v + 1.$$

Назначение дней l_v тоже допустимо: для ребра $u \rightarrow v$ имеем $h_u \geq h_v + 1$, откуда $l_u + 1 \leq l_v$.

Почему доступны все дни между границами

Зафиксируем вершину v и любой день $d \in [e_v, l_v]$. Положим $\Delta = d - e_v$ и назначим каждой вершине u день

$$c_u = \min(e_u + \Delta, l_u).$$

Для каждого ребра $u \rightarrow w$ выполняются оба неравенства $e_u + \Delta + 1 \leq e_w + \Delta$ и $l_u + 1 \leq l_w$. Взяв минимумы, получим $c_u + 1 \leq c_w$, то есть все зависимости сохраняются. При этом дни находятся между 1 и T , а $c_v = \min(d, l_v) = d$.

Таким образом, в допустимом диапазоне нет пропусков. Ответ для вершины:

$$l_v - e_v + 1 = T - h_v - e_v + 2.$$

Достаточно топологической сортировки и двух проходов по графу. Время и память — $O(n + m)$.

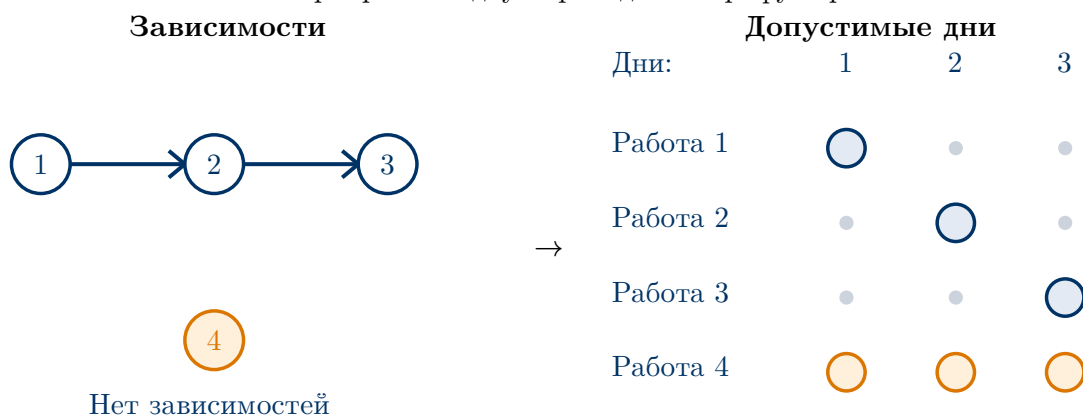


Рис. 3. Цепочка фиксирует дни работ 1, 2 и 3. Независимая работа 4 может стоять в любом из трёх дней.
Ответ: 1 1 1 3.

Е. Потеряшка

Из точки $(0, 0)$ бесконечно повторяется строка из n единичных шагов. Нужно найти первое касание стены ромба $|X| + |Y| = x$ или определить, что оно никогда не произойдёт.

Решение с бинарным поиском

Посчитаем координаты после каждого префикса строки: $(p_X[j], p_Y[j])$. Пусть (D_X, D_Y) — сдвиг за всю строку. Любой момент времени единственным образом представляется как $t = kn + j$, где $0 \leq j < n$. Координаты в этот момент:

$$(kD_X + p_X[j], kD_Y + p_Y[j]).$$

Зафиксируем j и найдём минимальное неотрицательное k , при котором расстояние $f_j(k) = |kD_X + p_X[j]| + |kD_Y + p_Y[j]|$ не меньше x . Если $f_j(0) \geq x$, подходит $k = 0$. Иначе можно сделать бинарный поиск: функция f_j выпукла, поэтому множество $k \geq 0$, при которых $f_j(k) < x$, является интервалом, содержащим ноль. После выхода из него вернуться внутрь нельзя. Само расстояние при этом не обязано возрастать с самого начала.

Если сдвиг ненулевой, то $|D_X| + |D_Y| \geq 1$, а длина префикса не больше n . По неравенству треугольника $f_j(k) \geq k - n$, поэтому для верхней границы поиска достаточно $k = x + n + 2$. Для всех j найдём первый опасный момент $kn + j$ и выберем наименьший. При $j = 0$ время 0 не будет выбрано: начальная точка строго внутри ромба. Если сдвиг нулевой, достаточно проверить один проход строки.

Время — $O(n \log(x + n))$, память — $O(n)$. Далее покажем, как убрать бинарные поиски и получить $O(n)$.

Улучшение до линейного времени: от ромба к квадрату

Введём координаты $u = X - Y$ и $v = X + Y$. Так как

$$|X| + |Y| = \max(|X - Y|, |X + Y|),$$

точка находится строго внутри ромба тогда и только тогда, когда

$$-x < u < x, \quad -x < v < x.$$

Теперь нужно следить за четырьмя обычными границами двух координат.

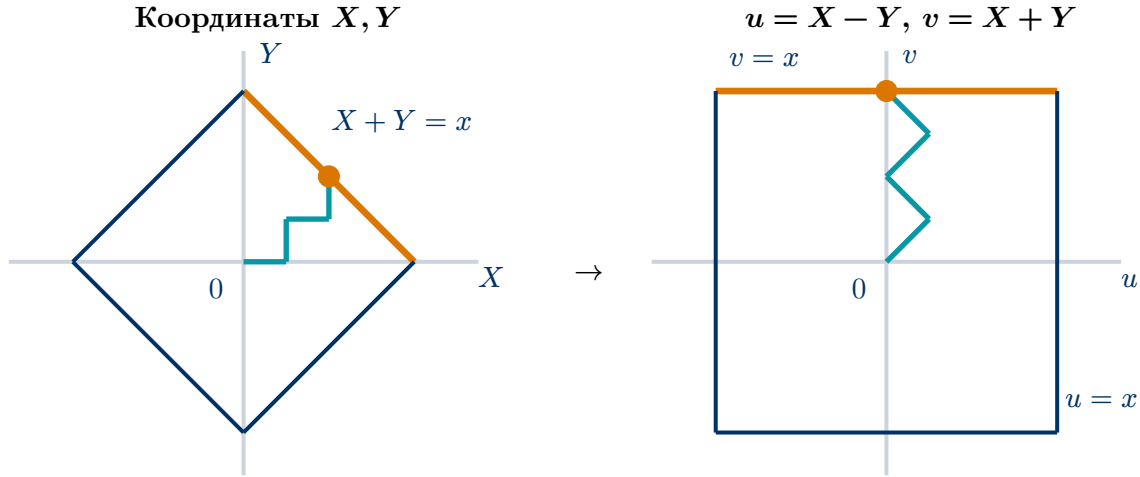


Рис. 4. При $x = 4$ путь RURU приводит к выходу $(2, 2)$, в новых координатах — $(0, 4)$. Оранжевая сторона $X + Y = x$ становится границей $v = x$.

Описываем один проход

Сначала выполним строку один раз. Если уже на этом проходе достигнута стена, сразу вернём первую такую точку. Иначе для каждой координаты $z \in \{u, v\}$ найдём минимум m_z и максимум M_z среди всех префиксов строки, включая пустой. Пусть (D_X, D_Y) — итоговое перемещение за строку. Тогда

$$\delta_u = D_X - D_Y, \quad \delta_v = D_X + D_Y.$$

Каждый новый проход повторяет ту же траекторию, только со сдвигом. После k полных проходов диапазон координаты z на очередном проходе равен

$$[k\delta_z + m_z, k\delta_z + M_z].$$

Пропускаем безопасные повторения

Если $\delta_z > 0$, нижняя граница больше не опасна, а верхняя впервые будет достигнута, когда $k\delta_z + M_z \geq x$. Если $\delta_z < 0$, аналогично следим за нижней границей. При нулевом сдвиге координата всегда повторяет уже проверенный безопасный диапазон. Поэтому индекс первого опасного прохода, считая с нуля, равен

$$K_z = \begin{cases} \left\lceil \frac{x - M_z}{\delta_z} \right\rceil & \delta_z > 0 \\ \left\lceil \frac{x + m_z}{-\delta_z} \right\rceil & \delta_z < 0 \\ \infty & \delta_z = 0. \end{cases}$$

Берём $K = \min(K_u, K_v)$. Если оба значения бесконечны, выхода нет. Иначе первые K проходов безопасны. Переносимся сразу в точку (KD_X, KD_Y) и выполняем строку ещё раз пошагово до первого касания стены. Именно этот последний проход определяет, какая из границ будет достигнута раньше и в какой точке.

Всего делаем не больше двух проходов по строке: время — $O(n)$, дополнительная память помимо строки — $O(1)$. Для перемещений и произведений используем 64-битные целые числа.

Г. Поиск составителей

Составитель i может сделать до x_i задач без напряжения и до y_i задач всего. Нужно получить ровно M задач, напрягая минимальное число составителей.

Без напряжения можно получить до $B = \sum_i x_i$ задач. Если $M \leq B$, ответ равен нулю: не обязательно использовать все возможности друзей. Если $M > \sum_i y_i$, задач не хватит даже при максимальной нагрузке, ответа нет.

В оставшемся случае нужно набрать ещё $R = M - B$ задач. Если напрячь составителя i , его дополнительная вместимость составит $c_i = y_i - x_i$. Значит, задача свелась к выбору минимального количества прибавок с суммой хотя бы R .

Отсортируем c_i по убыванию и будем брать их в этом порядке, пока сумма не достигнет R . Это оптимально: среди любых k составителей суммарная прибавка не больше суммы k наибольших c_i . Если даже этот префикс не покрывает недостачу, никакой другой набор из k человек не поможет.

Условие «ровно M » не мешает: последнюю выбранную прибавку можно использовать частично. Поэтому ответ — длина первого подходящего префикса.

Время — $O(n \log n)$, память — $O(n)$ для массива прибавок.

Г. Разбиение на отрезки

Массив натуральных чисел нужно разбить на подряд идущие непустые отрезки. Отрезок $[l, r]$ приносит $\left\lfloor \frac{a_r}{a_l} \right\rfloor$ очков. Требуется максимизировать их сумму.

Квадратичная динамика и лишние кандидаты

Пусть $\text{dp}[r]$ — лучший результат для первых r элементов, $\text{dp}[0] = 0$. Перебирая конец предпоследнего отрезка, получаем

$$\text{dp}[r] = \max_{0 \leq t < r} \left(\text{dp}[t] + \left\lfloor \frac{a_r}{a_{t+1}} \right\rfloor \right).$$

Прямой пересчёт занимает $O(n^2)$. Будем хранить кандидат t как пару $(a, b) = (a_{t+1}, \text{dp}[t])$. Его вклад при последнем элементе z равен $b + \left\lfloor \frac{z}{a} \right\rfloor$.

Заметим, что $\text{dp}[t+1] \geq \text{dp}[t] + 1$: к лучшему разбиению можно добавить одноэлементный отрезок. Поэтому у каждого нового кандидата b строго больше, чем у предыдущих. Если у старого кандидата ещё и a не меньше, чем у нового, старый заведомо хуже при любом положительном z : и свободный член меньше, и частное не больше. Его можно навсегда удалить.

После таких удалений кандидаты в стеке имеют строго возрастающие a и b . Но некоторые из них всё ещё могут никогда не давать оптимальный ответ.

От округлённых значений к прямым

Поскольку b целое, вклад кандидата — $\left\lfloor b + \frac{z}{a} \right\rfloor$. Для конечного набора чисел максимум коммутирует с округлением вниз:

$$\max_t \left\lfloor f_t(z) \right\rfloor = \left\lfloor \max_t f_t(z) \right\rfloor, \quad f_t(z) = b_t + \frac{z}{a_t}.$$

Таким образом, достаточно находить максимум обычных прямых f_t , а округлять только ответ. Это важно: сами округлённые функции не обязаны иметь одно строгое переключение победителя из-за возможных равенств.

У прямых наклоны $\frac{1}{a}$ убывают, а свободные члены b растут. Рассмотрим соседних кандидатов (a_1, b_1) и (a_2, b_2) , где $a_1 < a_2$, $b_1 < b_2$. Второй не хуже первого тогда и только тогда, когда

$$b_2 + \frac{z}{a_2} \geq b_1 + \frac{z}{a_1} \Leftrightarrow z \leq \frac{(b_2 - b_1)a_1 a_2}{a_2 - a_1}.$$

Значит, для целых z порог переключения равен

$$T = \left\lfloor \frac{(b_2 - b_1)a_1 a_2}{a_2 - a_1} \right\rfloor.$$

При $z \leq T$ выбираем второго кандидата, при $z > T$ — первого. На равенстве прямых договоримся выбирать более нового.

Стек порогов

Для каждого кандидата, кроме первого, храним порог, начиная с которого при движении от больших z к меньшим он заменяет предыдущего. У первого условно стоит $T_1 = +\infty$. Пороги должны строго убывать:

$$T_1 > T_2 > \dots > T_k.$$

Тогда кандидат i отвечает за диапазон $T_{i+1} < z \leq T_i$; последний — за все $z \leq T_k$.

При добавлении новой пары сначала удаляем с конца кандидатов с a не меньше нового. Затем считаем её пересечение с последним оставшимся кандидатом. Если новый порог не меньше порога последнего кандидата, у последнего не остаётся собственного диапазона: там, где он мог обгонять своего предшественника, новый уже не хуже него. Удаляем последнего и повторяем сравнение. Когда порядок порогов восстановлен, кладем новую пару в стек вместе с её порогом.

Получение ответа динамики

Перед вычислением $\text{dp}[r]$ добавим кандидата $(a_r, \text{dp}[r-1])$. Теперь в структуре представлены все допустимые начала последнего отрезка. Бинарным поиском найдём последний индекс с $T_i \geq a_r$ и для его пары (a, b) вычислим

$$\text{dp}[r] = b + \left\lfloor \frac{a_r}{a} \right\rfloor.$$

Каждый кандидат добавляется и удаляется не более одного раза, поэтому все изменения стека суммарно занимают $O(n)$. Запросов n , каждый за $O(\log n)$. Итого время — $O(n \log n)$, память — $O(n)$.

Пороги следует считать целочисленно, без `double`. Значения динамики храним в 64-битном типе, а произведение $(b_2 - b_1)a_1a_2$ вычисляем в `__int128`.

Н. Шахматные проказы

Нужно закодировать число от 1 до 20 000 восьмью не бьющими друг друга ладьями на доске 8×8 , а затем восстановить число после удаления любых двух ладей.

Что такое чётная перестановка

Расстановка восьми ладей задаётся перестановкой: p_i — столбец ладьи в строке i . Каждый столбец встречается ровно один раз.

Инверсия — это пара позиций $i < j$, для которой $p_i > p_j$: два элемента стоят в порядке, обратном возрастанию. Перестановка называется **чётной**, если число инверсий чётно, и **нечётной** иначе. Например, у $(2, 3, 1)$ две инверсии: число 2 стоит перед 1, и число 3 тоже стоит перед 1. Поэтому эта перестановка чётная. У $(2, 1, 3)$ одна инверсия, поэтому она нечётная. Ноль инверсий тоже соответствует чётной перестановке.

Обмен любых двух элементов меняет чётность. Для соседних элементов меняется только одна инверсия. Обмен элементов на расстоянии d можно выполнить за $2d - 1$ соседних обменов — нечётное число, поэтому чётность также изменится.

После удаления остаются два варианта

После удаления двух ладей известны шесть значений перестановки, две пустые строки r, s и два неиспользованных столбца a, b .

Есть ровно два способа дополнить расстановку: поставить ладьи в $(r, a), (s, b)$ или в $(r, b), (s, a)$. Эти перестановки отличаются обменом двух элементов, поэтому имеют разную чётность числа инверсий. Ровно одна из них чётная.



Рис. 5. Два дополнения расстановки отличаются одним обменом: одно чётное, другое нечётное.

Используем чётность как проверку

Договоримся кодировать числа только чётными перестановками. Тогда после удаления двух ладей декодер выбирает единственное чётное дополнение — исходную расстановку.

Чётных перестановок достаточно:

$$\frac{8!}{2} = 20\,160 \geq 20\,000.$$

Переберём все перестановки в лексикографическом порядке, оставим чётные и занумеруем их с единицы. При кодировании числа x выводим расстановку по x -й перестановке этого списка. При декодировании восстанавливаем чётное дополнение и находим его номер в том же списке. Таблица строится одинаково в обоих запусках, поэтому сохранять состояние между ними не требуется. Всего перестановок $8! = 40\,320$; даже прямой подсчёт инверсий за $O(8^2)$ для каждой вполне достаточен. Построение таблицы занимает $O(8! \cdot 8^2)$ времени и $O(8! \cdot 8)$ памяти — для фиксированной доски это небольшие постоянные величины.

I. Экзамен

Игроки по очереди берут карточку со стола и кладут в общую коробку. Проигрывает тот, после чьего хода НОД чисел в коробке стал равен 1, а также тот, кому не осталось хода. Можно выбрать, играть первым или вторым; нужно выбрать выигрышную сторону и реализовать её стратегию.

Почему достаточно количества и НОД

На первый взгляд нужно знать всё множество уже взятых карточек. На самом деле для исхода позиции достаточно двух величин: количества взятых карточек i и их НОД $g > 1$.

Обозначим через $\text{cnt}[g]$ число исходных карточек, делящихся на g . Все уже взятые числа кратны текущему НОД. Отсюда:

- На столе осталось ровно $\text{cnt}[g] - i$ кратных g . Взятие любой такой карточки сохраняет НОД, поэтому все эти ходы приводят к состоянию $(i + 1, g)$.
- Любая исходная карточка, не кратная g , точно ещё на столе. Её взятие меняет НОД на $h = \gcd(g, a_j) < g$. Значит, набор возможных уменьшений НОД тоже определяется одним g и исходным массивом.

Таким образом, какие именно кратные g уже взяты, неважно: доступные типы переходов полностью определяются парой (i, g) .

Динамика выигрышных позиций

Пусть $\text{win}[i][g]$ означает, что игрок, которому сейчас предстоит ход, может выиграть из этой позиции. При $i = n$ ходов нет, поэтому $\text{win}[n][g] = \text{false}$.

Из состояния (i, g) рассматриваем:

1. Переход в $(i + 1, g)$, если $\text{cnt}[g] > i$.
2. Для каждой исходной карточки, не кратной g , переход в $(i + 1, \gcd(g, a_j))$, если новый НОД больше 1.

Ходы с новым НОД 1 сразу проигрывают и не помогают сделать позицию выигрышной. Если есть хотя бы один безопасный переход в проигрышное состояние соперника, ставим $\text{win}[i][g] = \text{true}$. Если таких переходов нет, позиция проигрышная.

Считаем таблицу по убыванию i : каждый переход увеличивает число взятых карточек на единицу. Рассуждение о состоянии применяется к достижимым позициям; значения для невозможных сочетаний (i, g) на ответ не влияют.

Выбор стороны и восстановление ходов

Пустую коробку обработаем отдельно. Первый ход карточкой $a_j > 1$ оставляет сопернику состояние $(1, a_j)$. Если хотя бы для одной такой карточки $\text{win}[1][a_j] = \text{false}$, выбираем **First** и начинаем с неё. Иначе выбираем **Second**: любой безопасный первый ход соперника передаст нам выигрышную позицию, а небезопасный сразу завершит игру в нашу пользу.

Во время игры храним использованные индексы, количество взятых карточек и текущий НОД. На своём ходу перебираем оставшиеся карточки и выбираем ту, которая сохраняет НОД больше 1 и оставляет сопернику проигрышное состояние по таблице. Такой ход всегда существует: из проигрышной позиции соперник может либо проиграть сразу, либо передать нам выигрышную. Так как $a_i \leq n$, состояний $O(n^2)$, а переходы из каждого перебираются за $O(n)$. Заранее вычислим таблицу НОД для чисел от 1 до n и значения $\text{cnt}[g]$. Итого время — $O(n^3)$, память — $O(n^2)$. Восстановление всех ходов занимает ещё $O(n^2)$ времени и не меняет общую оценку.

Группировка переходов по делителям

Вместо перебора карточек можно сгруппировать переходы по новому НОД. Пусть $\text{count } [g][d]$ — число исходных карточек, для которых $\gcd(g, a_j) = d$. Для каждого делителя d числа g вычислим

$$\text{count } [g][d] = \text{cnt } [d] - \sum_{\substack{d|e, e|g \\ e>d}} \text{count } [g][e].$$

Делители обходим по убыванию, чтобы все вычитаемые слагаемые уже были известны. При $d < g$ эти карточки ещё не взяты, поэтому достаточно проверить $\text{count } [g][d] > 0$. После k ходов при $d = g$ остаётся $\text{cnt } [g] - k$ карточек. Это те же переходы, что в разборе, без повторного рассмотрения карточек с одинаковым результатом.

В интерактивной реализации выводим индекс карточки, а не её значение, сбрасываем буфер после каждого ответа и сразу завершаем при получении 0.

Ж. Тактический симулятор

Иерархия на подотрезке — декартово дерево по минимуму: корень соответствует наименьшему элементу, слева и справа рекурсивно строятся такие же деревья. Разрешённый тактический план — независимое множество вершин, то есть набор без двух вершин, соединённых ребром. Отрезок стабилен, если его корень входит хотя бы в одно независимое множество максимального размера.

Динамика независимого множества

Для вершины u обозначим её левого и правого ребёнка через p и q . Пусть $dp_1[u]$ — максимальный размер независимого множества с выбранной вершиной u , а $dp_0[u]$ — без неё. Тогда

$$dp_1[u] = 1 + dp_0[p] + dp_0[q],$$

$$dp_0[u] = dp[p] + dp[q],$$

$$dp[u] = \max(dp_0[u], dp_1[u]).$$

Для пустого дерева $dp = dp_0 = 0$. Отрезок стабилен тогда и только тогда, когда $dp_1[u] \geq dp_0[u]$. Равенство тоже подходит: нужен хотя бы один оптимальный план с выбранным корнем, а не обязательность корня во всех планах.

Введём разность $\Delta[u] = dp[u] - dp_0[u]$. Подставляя переходы, получаем

$$\Delta[u] = \max(0, 1 - \Delta[p] - \Delta[q]).$$

Следовательно, Δ всегда равна 0 или 1, причём $\Delta[u] = 1$ ровно тогда, когда у обоих детей $\Delta = 0$. Для пустого дерева полагаем $\Delta = 0$.

При этом

$$dp_1[u] - dp_0[u] = 1 - \Delta[p] - \Delta[q].$$

Поэтому отрезок нестабилен ровно тогда, когда у обоих его дочерних поддеревьев $\Delta = 1$. Будем считать такие отрезки, а в конце вычтем их число из $n^{\frac{n+1}{2}}$.

Зафиксируем минимум отрезка

Построим декартово дерево для всего массива монотонным стеком за $O(n)$. При обходе слева направо стек хранит правую ветку дерева, значения на ней возрастают. Для новой вершины снимаем с вершины стека элементы с большим значением; последний снятый становится её левым ребёнком. Если стек остался непустым, новая вершина становится правым ребёнком его вершины. После этого добавляем её в стек. Каждая вершина добавляется и снимается не более одного раза.

Поддерево вершины u занимает непрерывный диапазон $[L_u, R_u]$. Это максимальный диапазон вокруг позиции u , в котором a_u — минимум. Поэтому отрезок с минимумом в u имеет левую границу между L_u и u , а правую — между u и R_u .

Если обе стороны непусты, левая часть такого отрезка — суффикс левого поддерева, правая — префикс правого. Если одна из сторон пуста, её $\Delta = 0$, и отрезок заведомо стабилен.

Обозначим через S_u число непустых суффиксов диапазона поддерева u , для которых собственное декартово дерево имеет $\Delta = 1$. Аналогично P_u — число таких непустых префиксов. Каждый подходящий суффикс левого поддерева можно независимо сочетать с каждым подходящим префиксом правого. Число нестабильных отрезков с минимумом в u равно

$$S_u P_u.$$

Здесь важно различать Δ для полного поддерева и для выбранного суффикса или префикса: последние как раз учитываются величинами S и P .

Пересчёт суффиксов и префиксов

Пусть $\text{size } [u]$ — размер полного поддерева. Рассмотрим, где начинается суффикс диапазона поддерева u .

1. Если начало находится в правом поддереве q , вершина u в суффикс не входит. Это просто суффикс правого поддерева; подходящих вариантов S_q .
2. Если суффикс начинается в самой вершине u , его левая часть пуста, а правая — всё поддерево q . Он подходит при $\Delta[q] = 0$.
3. Если начало находится в левом поддереве p , суффикс содержит u , всё правое поддерево и суффикс левого. Для $\Delta = 1$ у корня обе дочерние части должны иметь $\Delta = 0$. Поэтому при $\Delta[q] = 1$ подходящих вариантов нет, а при $\Delta[q] = 0$ их $\text{size } [p] - S_p$.

Объединяя случаи, получаем

$$S_u = S_q + [\Delta[q] = 0] (\text{size } [p] + 1 - S_p).$$

Здесь скобка [условие] равна 1, если условие выполнено, и 0 иначе. Для префиксов рассуждение симметрично:

$$P_u = P_p + [\Delta[p] = 0] (\text{size } [q] + 1 - P_q).$$

Все величины считаем снизу вверх. Для пустого дерева $\text{size} = S = P = \Delta = 0$; для непустого дополнительно используем $\text{size } [u] = 1 + \text{size } [p] + \text{size } [q]$ и уже выведенный переход для $\Delta[u]$.

Итоговый ответ

$$\frac{n(n+1)}{2} - \sum_{u=1}^n S_{\text{left } (u)} P_{\text{right } (u)}.$$

Значения массива различны, поэтому у каждого отрезка единственный минимум. Каждый нестабильный отрезок попадёт ровно в одно слагаемое, а стабильные не попадут ни в одно. Это доказывает формулу.

Построение дерева и пересчёт всех величин занимают $O(n)$ времени и $O(n)$ памяти. Ответ и произведения считаем в 64-битном типе. Дерево может быть цепочкой длины n , поэтому обход снизу вверх удобно делать итеративно.

Ответ также можно суммировать по минимуму отрезка: для вершины u есть $(\text{size } [\text{left } (u)] + 1)(\text{size } [\text{right } (u)] + 1)$ способов выбрать две границы, из них вычитаются нестабильные. Сумма первых произведений равна $n \frac{n+1}{2}$, поэтому это та же формула ответа.